

Privacy-Preserving Electronic Voting Using Zero-Knowledge Proofs on a Custom Blockchain

Abstract—Electronic voting systems must simultaneously guarantee eligibility verification, ballot privacy, end-to-end verifiability, and coercion resistance. We present a production-grade e-voting platform that achieves *voter–ballot unlinkability* through zero-knowledge proofs on a purpose-built, gasless blockchain. A Noir arithmetic circuit over the BN254 scalar field proves voter membership in a Merkle commitment tree via the Poseidon hash, verified on-chain by the UltraHonk proof system—without revealing voter identity. Authentication employs a three-factor protocol (OTP, biometric, hardware-backed PIN) on a native mobile application; the ballot is then cast from an ephemeral wallet, making it cryptographically unlinkable to the registered identity. The system models Sri Lanka’s 160-division electorate with village-officer enrollment, on-chain national aggregation, and a custom Go blockchain embedding the Ethereum Virtual Machine—processing all transactions at zero cost. On-chain proof verification completes in under 5 ms; on-device proof generation takes 15–30 s via WebAssembly. Any observer can independently verify every proof and recompute the tally from public state.

Index Terms—Zero-knowledge proofs, electronic voting, blockchain, anonymous credentials, Poseidon hash, Merkle tree, UltraHonk.

I. INTRODUCTION

Democratic elections require a balance between *accountability*—only eligible citizens vote, each at most once—and *privacy*—no party learns how any individual voted. Paper-ballot systems achieve isolation through physical means but scale poorly and resist auditing.

Electronic voting introduces new threats: server-side manipulation, identity–ballot linkage via metadata, and receipt-based coercion. Prior blockchain approaches [1], [2], built on the ledgers introduced by Bitcoin [14] and generalized by Ethereum [15], improved auditability but left the sender address as a linkable pseudonym.

Our design replaces *institutional trust* with *cryptographic unlinkability*. A registered voter holds a hiding commitment in a public Merkle tree; at voting time, the voter proves membership in zero knowledge [16] and casts from an ephemeral wallet. Formally:

$$\Pr[A(\text{chain}) \rightarrow (\text{Voter}, \text{Ballot})] \leq \frac{1}{|\mathcal{V}|} + \text{negl}(\lambda), \quad (1)$$

where $\mathcal{V}$ is the voter set, λ the security parameter, and negl negligible—the adversary gains no advantage over random guessing.

Contributions.

- 1) A ZK circuit achieving voter–ballot unlinkability via Merkle-membership proofs and burner-wallet casting.

- 2) A three-factor mobile authentication protocol binding a hardware-resident key to a physically present voter.
- 3) A custom, gasless Go blockchain embedding the EVM, providing full sovereignty at zero cost to participants.
- 4) A multi-division architecture with village-officer enrollment and on-chain national aggregation for 160 divisions.
- 5) End-to-end verifiability: any observer recomputes the tally from public state.

II. RELATED WORK

A. Cryptographic Voting Protocols

Helios [5] demonstrated verifiable web voting using ElGamal and a bulletin board, but relies on trusted trustees for decryption. Mix-net designs, originating with Chaum [18], break the sender–ballot link at the cost of expensive shuffle proofs. Our system eliminates trustees: privacy derives from the voter’s own zero-knowledge proof.

B. Blockchain-Based Voting

McCorry *et al.* [1] presented the first Ethereum voting contract but its $O(n^2)$ complexity does not scale nationally. Voatz [3] moved to mobile but the audit [4] exposed linkability and centralized trust—exactly what we eliminate.

C. Anonymous Membership Primitives

Semaphore [9] provides anonymous signaling via Merkle proofs but is a general gadget, not an election platform. MACI [8] adds collusion resistance but reintroduces a trusted coordinator. Our system retains Semaphore-style unlinkability, removes the coordinator, and provides a complete administrative platform with hardware-backed authentication (Table VI).

III. PRELIMINARIES

A. Notation

We work over $\mathbb{F}_p$ (BN254, $p \approx 2^{254}$). Poseidon [12] is our hash: algebraically friendly, minimizing circuit constraints. Poseidon_{*k*} denotes the *k*-arity variant.

Definition 1 (Voter Commitment). Given random nullifier $r \in \mathbb{F}_p$ and secret $s \in \mathbb{F}_p$:

$$\text{cm} = \text{Poseidon}_2(r, s), \quad \text{nh} = \text{Poseidon}_1(r). \quad (2)$$

This commitment–nullifier construction follows the Zero-cash paradigm [19], where a hiding commitment authorizes an action and a deterministic nullifier prevents its reuse.

TABLE I
SECURITY PROPERTIES

Property	Guarantee
Eligibility	Only tree members produce valid ballots
Unlinkability	Ballot $\not\rightarrow$ voter (Eq. 1)
One-person-one-vote	Duplicate nullifier rejected on-chain
E2E verifiability	Tally recomputable from public state
Coercion resistance	No transferable receipt

B. Merkle Accumulator

Commitments are accumulated in a Merkle tree [17]—specifically a Lean Incremental Merkle Tree [13]—of depth $d \leq 16$. For leaf index i with path $\pi = (\sigma_1, \dots, \sigma_d)$:

$$h_0 = \text{cm}, \quad h_j = \begin{cases} \text{Poseidon}_2(h_{j-1}, \sigma_j) & b_j = 0 \\ \text{Poseidon}_2(\sigma_j, h_{j-1}) & b_j = 1 \end{cases} \quad (3)$$

where b_j is the j -th bit of i ; root $R = h_d$.

C. Threat Model

Assumptions: (i) honest-majority consensus; (ii) sound ZK proof system; (iii) hardware keystore integrity; (iv) adversary controls network and servers but not validator majority. Target properties: Table I.

IV. SYSTEM ARCHITECTURE

A. Custom Go Blockchain

The production backend is a purpose-built blockchain in Go, embedding go-ethereum’s Ethereum Virtual Machine [15]. It executes identical Solidity bytecode, preserving all smart-contract guarantees—deterministic execution, state immutability, and auditability—under the election commission’s sovereign control.

The chain is deployed across geographically distributed validator nodes with Practical Byzantine Fault Tolerant (PBFT) consensus [20] ensuring finality under partial failure. It is **gasless**: no fee market, no cryptocurrency, zero economic barrier. The node exposes JSON-RPC and REST APIs.

B. Smart-Contract Layer

- **Voting.sol** — Per-division state machine (Setup→Registration→Voting→Ended). Holds the GN-managed address *allowlist*, the commitment Merkle tree, and the used-nullifier set; enforces registration and voting checks (Section IX-B).
- **HonkVerifier.sol** — Auto-generated UltraHonk verifier; called *internally* by Voting.sol to check each ballot proof.
- **ElectionRegistry.sol** — Factory that deploys per-division Voting contracts and aggregates national results via pure view functions.
- **NicRegistry.sol** — Independent registry storing hashed NICs; the GN officer calls `reserveNicHash` to guarantee each citizen enrolls at most once.

C. Application Layer

The *Election Authority* and *Village Officers* (Sinhala: *Grama Niladhari*) use a Next.js web console. *Voters* use a React Native mobile app storing keys in hardware. *Observers* access a read-only audit interface.

V. ZERO-KNOWLEDGE CIRCUIT

A. The NP Relation

Public: $x = (\text{nh}, R, v, d)$. Witness: $w = (r, s, i, \pi)$. The circuit proves:

$$\mathcal{R} = \left\{ (x, w) \left| \begin{array}{l} \text{nh} = \text{Poseidon}_1(r) \\ \text{cm} = \text{Poseidon}_2(r, s) \\ \text{Root}(\text{cm}, i, \pi) = R \\ 0 \leq v < 2^8 \end{array} \right. \right\} \quad (4)$$

B. Proof System

We adopt UltraHonk [11], a successor to PLONK [7] that—unlike earlier pairing-based SNARKs such as Groth16 [6]—requires no per-circuit trusted setup. It uses a keccak transcript for cheap on-chain Fiat–Shamir, and verification is constant in witness size.

C. Security Properties

Property 1 (Soundness). Valid proof $\Rightarrow$ prover knows (r, s) with $\text{Poseidon}_2(r, s)$ in the tree.

Property 2 (Zero-Knowledge). Proof reveals nothing beyond truth of (4); verifier learns membership but not *which* leaf.

Property 3 (Double-Vote Prevention). Nullifier set N on-chain: accept iff $\text{nh} \notin N$; then $N \leftarrow N \cup \{\text{nh}\}$.

D. Circuit Implementation

Listing 1 presents the circuit in Noir [10], a domain-specific language for SNARK proving. The public input `vote` is bound inside the proof, preventing an adversary from replacing the ballot in transit (front-running); the range constraint `vote < 28` rejects malformed candidate encodings.

```
fn main(
  nullifier_hash: pub Field, // nh
  root: pub Field, // R
  vote: pub Field, // v
  depth: pub u32, // d
  nullifier: Field, // r
  secret: Field, // s
  index: Field, // i
  siblings: [Field; 16], // path
) {
  // 1. Nullifier binds identity, hides it
  assert(hash_1([nullifier]) == nullifier_hash);
  // 2. Commitment = Poseidon(nullifier, secret)
  let cm = hash_2([nullifier, secret]);
  // 3. Merkle membership: recompute root
  let r = compute_root(cm, index, depth, siblings);
  assert(r == root);
  // 4. Range check on the candidate id
  vote.assert_max_bit_size::<8>();
}
```

Listing 1. Noir circuit enforcing relation (4).

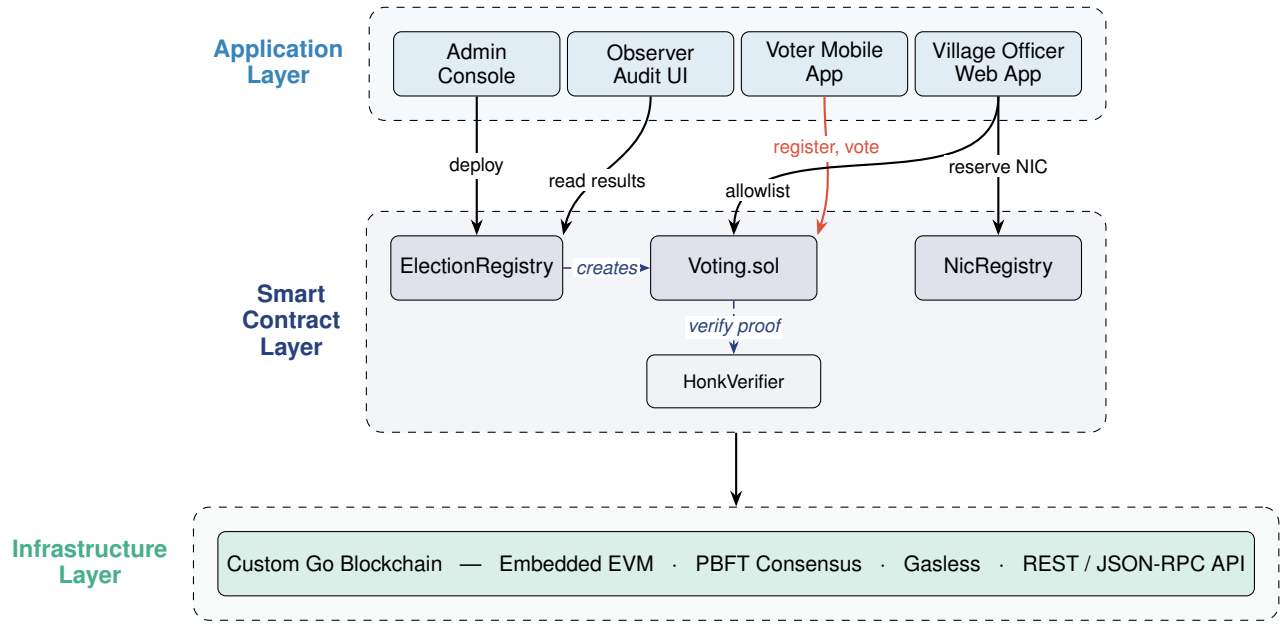


Fig. 1. System architecture. The admin deploys divisions via `ElectionRegistry` (factory); the village officer allowlists voter addresses on `Voting.sol` and reserves the voter’s NIC hash on `NicRegistry`; the voter registers a commitment and later casts a ballot on `Voting.sol`, which internally invokes `HonkVerifier`. Observers read aggregated results. All contracts run on the custom gasless Go blockchain.



Fig. 2. Election phase automaton. Each transition is authority-triggered; out-of-phase contract calls revert.

Algorithm 1 Election Setup (Authority)

- 1: Deploy `ELECTIONREGISTRY(admin, verifier)` and `NICREGISTRY(admin)`
- 2: **for** each division $d_j \in D$ **do**
- 3: `ELECTIONREGISTRY.CREATEDIVISION( $d_j$ )` $\triangleright$ factory deploys `VOTING`
- 4: `VOTING[ $d_j$ ].SETCANDIDATES( $C$ ); SETGNOFFICER( $gn_j$ )`
- 5: `NICREGISTRY.SETVOTINGCONTRACT( $d_j$ , true)`
- 6: **end for**
- 7: Open the Registration phase on every division

VI. PROTOCOL SPECIFICATION

Three algorithms formalize the protocol:

The crux is key separation: the voter is allowlisted and registers a commitment from their *identity* address (verified in person), but casts the ballot from an unrelated *burner* key k' . The ZK proof bridges eligibility across this gap without revealing the link.

VII. VOTER AUTHENTICATION

Let $g_{otp}, g_{bio}, g_{pin} \in \{0, 1\}$. The keystore unlocks iff:

$$\text{Unlock} = 1 \iff g_{otp} \wedge g_{bio} \wedge g_{pin}. \quad (5)$$

Algorithm 2 Voter Enrollment (Village Officer + Voter)

- 1: **Voter device** (secure element):
- 2: Sample $r, s \leftarrow \mathbb{F}_p$; $cm \leftarrow \text{Poseidon}_2(r, s)$
- 3: Store (r, s) non-extractably; show voter address + cm
- 4: **Village Officer** (in person, after NIC check):
- 5: `NICREGISTRY.RESERVENICHASH( $\mathcal{H}(\text{NIC}), d$ )`
- 6: **revert** if NIC already reserved $\triangleright$ one person, once
- 7: `VOTING[ $d$ ].ADDVOTERS([voterAddr], [true])` $\triangleright$ allowlist
- 8: **Voter** (from own allowlisted address):
- 9: `VOTING[ $d$ ].REGISTER( $cm$ )`
- 10: **require** allowlisted $\wedge$ not yet registered
- 11: **require** cm unused; insert leaf into tree

Algorithm 3 Anonymous Ballot Casting (Voter)

- 1: **Authenticate:** $\text{OTP} \wedge \text{biometric} \wedge \text{PIN} \Rightarrow$ unlock keystore
- 2: Retrieve (r, s) ; fetch root R and path (i, π)
- 3: $nh \leftarrow \text{Poseidon}_1(r)$; $x \leftarrow (nh, R, v, d)$; $w \leftarrow (r, s, i, \pi)$
- 4: $\pi_{zk} \leftarrow \text{Prove}(x, w)$ $\triangleright$ on-device, 15–30 s
- 5: $k' \leftarrow \{0, 1\}^{256}$ $\triangleright$ fresh ephemeral (burner) key
- 6: Submit `VOTING[ $d$ ].VOTE( $\pi_{zk}, nh, R, v, \text{depth}$ )` from `addr( $k'$ )`
- 7: **On-chain:** require R current; require v valid
- 8: require $\text{Verify}(x, \pi_{zk})$; require $nh \notin N$
- 9: $N \leftarrow N \cup \{nh\}$; `tally[ $v$ ] += 1`

Each factor defeats an independent threat:

- **OTP** — phone is present (defeats delegation/proxy).
- **Biometric** — enrolled body present (defeats theft).

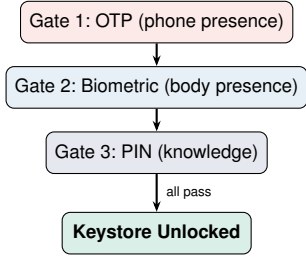


Fig. 3. Three-factor authentication. All gates must pass before the hardware keystore releases cryptographic material.

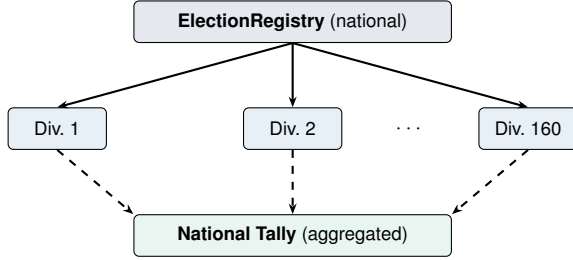


Fig. 4. Division hierarchy. The registry deploys 160 independent division contracts (solid) and aggregates their counts into the national tally (dashed).

- **PIN** — knowledge factor (fallback; anti-coercion).

Keys are generated in the device’s Trusted Execution Environment [21] (Android Keystore / iOS Secure Enclave)—non-extractable even on rooted devices.

A. Ephemeral Submission

The gasless chain eliminates the “funding trace” attack: on gas-based chains, the ETH source for a burner wallet can be traced back. Our burner k' submits directly via REST at zero cost—no funding link exists.

VIII. MULTI-DIVISION ARCHITECTURE

Sri Lanka’s electoral system assigns a *Village Officer* (Sinhala: *Grama Niladhari*; GN) to each division as the first point of citizen identity verification. Our system maps this directly:

- The authority binds a GN officer’s address to each division.
- After an in-person NIC check, the GN reserves the hashed NIC and allowlists the voter’s address.
- The voter then registers a commitment from that allowlisted address.

National tally for candidate c :

$$T(c) = \sum_{j=1}^N \mathbf{1}[\text{div}_j \text{ active}] \cdot t_j(c), \quad (6)$$

computed as a pure view function—any observer verifies independently.

IX. IMPLEMENTATION

A. Technology Stack

The system is implemented as a monorepo spanning circuits, contracts, web, and mobile (Table II). All components are open

TABLE II
IMPLEMENTATION STACK

Component	Technology
ZK circuit	Noir 1.0 + Barretenberg (UltraHonk)
Smart contracts	Solidity 0.8.30 (EVM Paris)
Merkle tree	LeanIMT + PoseidonT3 (on-chain)
Blockchain	Custom Go node (embedded go-ethereum)
Web console	Next.js 15, wagmi, viem
Mobile app	React Native (Expo SDK 54)
Key storage	Expo SecureStore (HW keystore)
Proving (mobile)	bb.js WebAssembly

TABLE III
ON-CHAIN ANTI-FRAUD CHECKS

Check	State	Prevents
Hashed-NIC unique	usedNicHashes	Same person enrolled twice
Address allowlisted	s_voters	Non-approved registration
One reg. per address	s_hasRegistered	Re-registration by a voter
Commitment unique	s_commitments	Duplicate leaf insertion
Nullifier unique	s_nullifierHashes	Double voting

and reproducible; the same Solidity bytecode is executed by both the development network and the production Go chain.

B. Layered Fraud Prevention

The system rejects fraudulent participation at *four* independent layers, each enforced on-chain (Table III). The first three gate *registration* (one real citizen → one commitment); the fourth gates *voting* (one commitment → one ballot).

Listing 2 shows the registration guards, and Listing 3 the nullifier-guarded vote. Together they enforce all four layers.

```

function register(uint256 commitment)
    external inPhase(Phase.Registration) {
        // (1) must be allowlisted, (2) not yet registered
        if (!s_voters[eid][msg.sender] ||
            s_hasRegistered[eid][msg.sender])
            revert Voting__NotAllowedToVote();
        // (3) commitment must be fresh
        if (s_commitments[eid][commitment])
            revert Voting__CommitmentAlreadyAdded(commitment);
        s_commitments[eid][commitment] = true;
        s_hasRegistered[eid][msg.sender] = true;
        tree.insert(commitment); // add Merkle leaf
    }
  
```

Listing 2. Registration guards (Solidity).

```

function vote(bytes proof, bytes32 nullifierHash,
    bytes32 root, bytes32 vote, bytes32 depth)
    external inPhase(Phase.Voting) {
        require(root == tree.root(), "stale root");
        bytes32[] memory pub = [nullifierHash,
            root, vote, depth];
        require(verifier.verify(proof, pub), "bad proof");
        // reject a reused nullifier => no double vote
        if (s_nullifierHashes[eid][nullifierHash])
            revert Voting__NullifierHashAlreadyUsed(
                nullifierHash);
        s_nullifierHashes[eid][nullifierHash] = true;
    }
  
```

```

voteCounts[eid][uint256(vote)] += 1;
}

```

Listing 3. Nullifier-guarded vote (Solidity).

C. Worked Example

Consider voter Alice in division *Kaduwela*. After verifying her NIC in person, the village officer reserves $\mathcal{H}(\text{NIC}_{\text{Alice}})$ on `NicRegistry` (a second attempt for Alice anywhere in the country reverts) and allowlists her address on `Voting`. Alice’s device samples r, s and registers leaf $\text{cm} = \text{Poseidon}_2(r, s)$ from her own allowlisted address; the contract confirms she is allowlisted, has not registered before, and that cm is fresh, then inserts it at index $i=42$. At voting time, after passing the three gates, her app computes $\text{nh} = \text{Poseidon}_1(r)$ and a proof π_{zk} for candidate $v=3$, submitted from a fresh burner key k' . The contract checks the root, verifies the proof in <5 ms, confirms $\text{nh} \notin N$, and increments `tally[3]`. Observers see only that *some* valid *Kaduwela* voter chose candidate 3; Alice’s identity, index 42, and path stay hidden. Any second ballot by Alice reproduces the same nh and is rejected.

X. SECURITY ANALYSIS

Theorem 1 (Voter–Ballot Unlinkability). *For any PPT adversary $\mathcal{A}$ observing the public chain, the probability of linking a ballot to its voter satisfies (1).*

Proof sketch. Public artifacts: (nh, R, v, d) , proof π_{zk} , sender $\text{addr}(k')$. By ZK, π_{zk} is simulatable and leaks no index. Under Poseidon one-wayness, nh is independent of cm . Key k' is fresh with no funding trace (gasless). Hence $\mathcal{A}$ can only guess uniformly. $\square$

Lemma 1 (Tally Integrity). *No adversary can inflate a count without forging a proof (contradicting soundness) or reusing a nullifier (rejected on-chain).*

A. Coercion Resistance and Receipt-Freeness

A voting system is *receipt-free* if a voter cannot prove to a third party how they voted, even if willing. Our design provides receipt-freeness against a passive coercer: the ZK proof π_{zk} attests only to *membership*, not identity, so a coerced voter cannot demonstrate that a particular on-chain ballot is theirs. The ephemeral key k' is discarded after submission, leaving no signing artifact to surrender. The PIN gate additionally enables a duress fallback: under observation, the voter authenticates normally while retaining plausible deniability, since no observer can distinguish a genuine from a coerced ballot on-chain. We note that an *active* coercer who physically supervises the entire session and seizes the device mid-protocol remains outside our threat model, as it does for all remote-voting systems.

XI. EVALUATION

A. Deployment

The custom Go blockchain runs on distributed validator nodes with PBFT consensus. Benchmarks simulate 10,000 voters per division across 160 parallel divisions. Client devices: Pixel 7 Pro (Android 14), Samsung Galaxy A14 (Android 13).

TABLE IV
ATTACK–DEFENSE MAPPING

Attack	Defense
De-anonymization	ZK + burner (Thm. 1)
Double voting	Nullifier set (Property 3)
Duplicate enrollment	Hashed-NIC + <code>hasRegistered</code>
Ineligible voting	Allowlist + Merkle soundness
Ballot stuffing	Commitment uniqueness
Vote substitution	Vote is public input
Device theft	Biometric + PIN (Eq. 5)
Proxy voting	OTP (phone presence)
Funding trace	Gasless chain
Server tampering	Immutable on-chain state
Coercion	No transferable receipt

TABLE V
END-TO-END PERFORMANCE

Operation	Latency
Voter enrollment	<1 s
Proof generation (mobile WASM)	15–30 s
Proof generation (native)	3–8 s
On-chain proof verification	<5 ms
Vote tx finality (PBFT)	<2 s
National aggregation (160 divs.)	<500 ms
Node throughput	>200 votes/s

B. Complexity

Proof size and verification are *constant* in electorate size:

$$|\pi_{\text{zk}}| = O(1), \quad T_{\text{verify}} = O(1), \quad |\mathcal{C}| = O(\log |\mathcal{V}|). \quad (7)$$

C. Performance

Each verification uses <5 ms CPU; a single node handles >200 votes/second. With 160 divisions on parallel nodes, the system comfortably serves Sri Lanka’s ~ 16 M voters within a standard polling window.

D. Scalability Projection

Because verification cost is constant (7), aggregate capacity scales linearly with node count. At a sustained 200 votes/s per node, a single node clears 7.2×10^5 ballots/hour; the full 16 M electorate is processed in under 23 node-hours, or well under two hours across a modest cluster of a dozen nodes—comfortably inside a one-day poll. Storage is dominated by the nullifier set: at 32 bytes per spent nullifier, 16 M ballots occupy ≈ 512 MB, trivially held in node memory. The Merkle depth $d=16$ supports 65,536 voters per division; larger divisions simply increase d by one per doubling, adding one Poseidon hash to the circuit.

E. Deployment Considerations

A production rollout requires: (i) a multi-party *trusted-setup ceremony* for the structured reference string, with independent auditors so no single party holds a trapdoor; (ii) geographically replicated validator nodes under PBFT for Byzantine fault tolerance; (iii) a hardened key-provisioning flow binding each

TABLE VI
COMPARISON WITH EXISTING SYSTEMS

	Ours	MACI	Sem.	Voatz
ZK anonymity	✓	✓	✓	×
No coordinator	✓	×	✓	×
Gasless	✓	×	×	✓
Custom chain	✓	×	×	×
Multi-division	✓	×	×	partial
HW-backed auth	✓	×	×	✓
On-device proof	✓	×	×	×
E2E verifiable	✓	✓	✓	×

voter’s secure-element key to an in-person identity check by the village officer; and (iv) public disclosure of contract bytecode and circuit artifacts so that any citizen can independently reproduce verification. These are operational—not architectural—requirements; the cryptographic core is unchanged.

F. Comparison

Table VI contrasts our system with representative prior work. Ours is the only design that is simultaneously gasless, deployed on a sovereign custom chain, division-aware, and hardware-authenticated while preserving ZK anonymity and end-to-end verifiability.

XII. CONCLUSION

We presented a production-grade ZK e-voting system on a custom gasless blockchain. It achieves unlinkability (1) via Merkle proofs and ephemeral casting, enforces one-person-one-vote via deterministic nullifiers, and provides E2E verifiability through publicly recomputable tallies (6). Three-factor hardware authentication (5) binds ballots to present voters without creating linkable traces. The gasless chain eliminates all economic barriers while preserving smart-contract guarantees, and the constant-cost verifier (7) scales to a 16-million-voter electorate. The 160-division architecture maps faithfully onto Sri Lanka’s administrative structure, demonstrating that privacy-preserving e-voting is achievable at national scale without trusted intermediaries. By replacing institutional trust with cryptographic guarantees that every citizen can independently verify, the system offers a concrete path toward auditable, private, and inclusive digital democracy.

REFERENCES

- [1] P. McCorry, S. F. Shahandashti, and F. Hao, “A smart contract for boardroom voting with maximum voter privacy,” in *Proc. FC*, 2017, pp. 357–375.
- [2] Agora, “Bringing our elections into the 21st century,” White Paper, 2017.
- [3] Voatz Inc., “Mobile voting platform,” Tech. Report, 2018.
- [4] M. Specter, J. Koppel, and D. Weitzner, “The ballot is busted before the blockchain: A security analysis of Voatz,” in *Proc. USENIX Security*, 2020.
- [5] B. Adida, “Helios: Web-based open-audit voting,” in *Proc. USENIX Security*, 2008.
- [6] J. Groth, “On the size of pairing-based non-interactive arguments,” in *Proc. EUROCRYPT*, 2016.
- [7] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, “PLONK,” ePrint 2019/953.
- [8] Ethereum Foundation, “Minimum Anti-Collusion Infrastructure (MACI),” 2020.
- [9] Privacy & Scaling Explorations, “Semaphore,” 2020.
- [10] Aztec Labs, “Noir: A DSL for SNARK proving,” 2023.
- [11] Z. J. Williamson *et al.*, “Honk: Recursive proof composition,” Aztec Labs, 2023.
- [12] L. Grassi *et al.*, “Poseidon: A new hash function for ZK proof systems,” in *Proc. USENIX Security*, 2021.
- [13] Privacy & Scaling Explorations, “Lean Incremental Merkle Tree,” zk-kit, 2023.
- [14] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” White Paper, 2008.
- [15] G. Wood, “Ethereum: A secure decentralised generalised transaction ledger,” *Ethereum Project Yellow Paper*, 2014.
- [16] S. Goldwasser, S. Micali, and C. Rackoff, “The knowledge complexity of interactive proof systems,” *SIAM J. Comput.*, vol. 18, no. 1, pp. 186–208, 1989.
- [17] R. C. Merkle, “A digital signature based on a conventional encryption function,” in *Proc. CRYPTO*, 1987, pp. 369–378.
- [18] D. Chaum, “Untraceable electronic mail, return addresses, and digital pseudonyms,” *Commun. ACM*, vol. 24, no. 2, pp. 84–90, 1981.
- [19] E. Ben-Sasson *et al.*, “Zerocash: Decentralized anonymous payments from Bitcoin,” in *Proc. IEEE S&P*, 2014, pp. 459–474.
- [20] M. Castro and B. Liskov, “Practical Byzantine fault tolerance,” in *Proc. OSDI*, 1999, pp. 173–186.
- [21] GlobalPlatform, “TEE system architecture,” Technical Specification, 2018.